פורמט לתיעוד — גרסה 0.6.0

קפסה

פורמט אחד לתיעוד **פרויקטים**, **צוות** ו**חברה** — כקבצים קריאים שחיים בתוך המאגר עצמו, ובנוי כך שאפשר לשלוף ממנו בדיוק את החלק הרלוונטי למשימה אחת, ולא את הכול.

1 מה זו קפסה, ואיזו בעיה היא פותרת

2 הרשומה — היחידה שהכול בנוי ממנה

3 מעץ לגרף: הכלה וקשתות

4 מה מתועד בפרויקט

5 בניית הקשר לסוכן למשימה אחת

6 הצוות והחברה: אנשים, תפקידים ותובנות

7 היחס בין תיעוד החברה לתיעוד הפרויקטים

8 מה נבדק אוטומטית — ומה קפסה איננה

מה זו קפסה, ואיזו בעיה היא פותרת

בכל פרויקט מצטברת אמת שאינה קוד: מה נדרש, מה הוחלט ולמה, מה נשבר, מה נלמד. בדרך כלל היא מפוזרת — חלקה בכלי-משימות, חלקה בצ'אט, ורובה בראש של מי שהיה שם. כשהאדם עוזב, או כשעוברת שנה, היא נעלמת.

קפסה היא תיקייה של קבצי טקסט קריאים שיושבת בתוך המאגר של המוצר. כל עובדה דורבנית היא רשומה — קובץ אחד, שאדם יכול לקרוא ומכונה יכולה לנתח. אין שרת, אין מסד-נתונים, ואין תוכנה שצריכה לרוץ כדי שהתיעוד יהיה קיים: מעתיקים את הפרויקט — התיעוד בא איתו; מוחקים כל כלי — התיעוד נשאר שלם.

שלוש התכונות שקובעות הכול

שלוש התכונות שכל השאר נגזר מהן

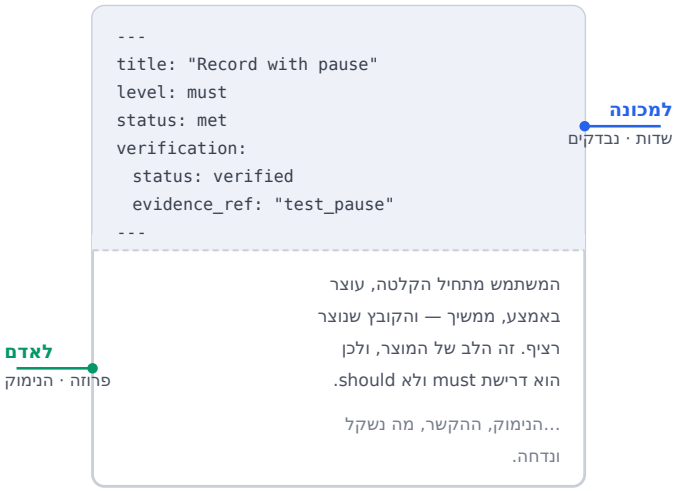
למה זה חשוב במיוחד כשסוכני-AI עובדים על הפרויקט

לסוכן יש חלון-הקשר מוגבל. אם התיעוד הוא ערימה אחת, יש שתי אפשרויות גרועות: להזרים לו הכול — יקר, איטי, ומטביע את הרלוונטי ברעש — או לתת לו מעט מדי, ואז הוא מנחש. קפסה בנויה כדי לאפשר אפשרות שלישית: **לשלוף בדיוק את השכונה של מה שהוא עובד עליה**. פרק 5 מראה איך.

> **הבחנה שכדאי להחזיק לאורך המסמך:** קפסה מתעדת **אמת דורבנית** — מה נכון לגבי הפרויקט. היא **לא** מתעדת מצב-ריצה: מי עובד עכשיו, כמה זה עלה, מה קורה ברגע זה. זה נכון לרגע וזורק אחריו; הקפסולה נועדה להיקרא בעוד שלוש שנים.

הרשומה — היחידה שהכול בנוי ממנה

כל דבר בקפסה הוא רשומה: קובץ אחד, עם שני חלקים. למעלה שדות מסודרים שמכונה קוראת; למטה טקסט חופשי שאדם קורא. קובץ אחד משרת את שניהם — ולכן אין שתי גרסאות שיכולות לסתור זו את זו.



רשומה אחת — למעלה מה שנבדק, למטה מה שמסבירים

ההפרדה הזאת היא הרעיון, לא עיצוב

שימו לב לשדה `status: met` — "הדרישה מתקיימת". בקפסה אי אפשר לכתוב אותו לבד: הוא מחייב בלוק `verification` שמצביע על **הראיה** — הבדיקה שרצה, הסריקה, המדידה. טענה שאין מתחתיה ראיה נחשבת "לא מאומתת", וזו עובדה גלויה ולא "כנראה בסדר".

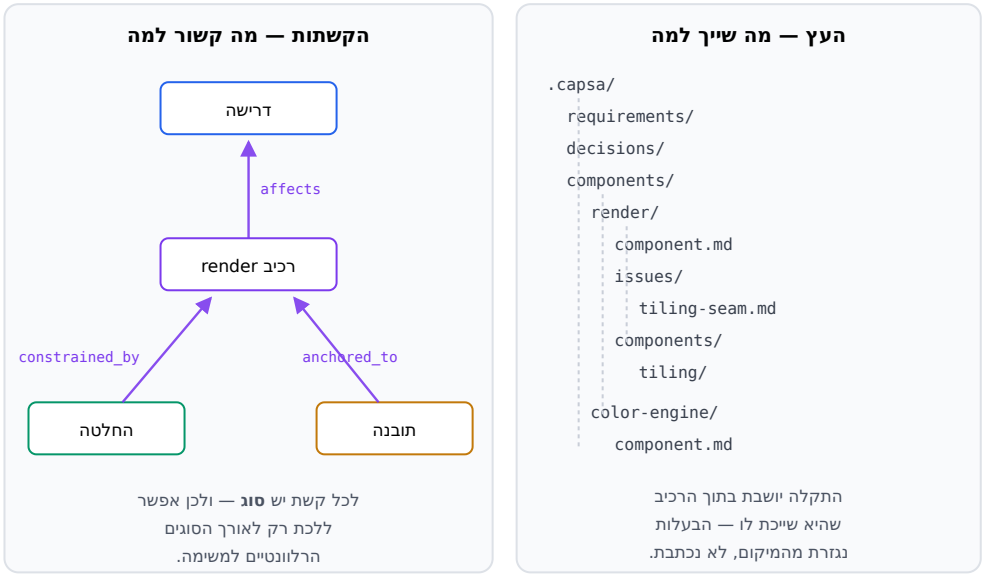
הכלל הזה הוא מה שמבדיל בין תיעוד שאפשר לסמוך עליו לבין תיעוד שמישהו כתב פעם. הוא גם מה שמאפשר לשאול שאלות במכונה: *"האם כל דרישת-חובה מאומתת לפני שחרור?"* היא שאילתה, לא סקר.

זהות: הנתיב הוא השם

רשומה מזוהה לפי **הנתיב שלה** — התיקייה והשם. אין מספר רץ מרכזי שצריך להקצות. זה נשמע טכני, והוא פותר בעיה מאוד לא-טכנית: כששני אנשים עובדים במקביל, מספור רץ מבטיח שהם יתנגשו — שניהם ייקחו את "המספר הבא". נתיב לא מתנגש.

מעץ לגרף: הכלה וקשתות

לקפסה יש שני מבנים בו-זמנית, בדיוק כמו לקוד: **עץ** שאומר מה שייך למה, ו**קשתות** שאומרות מה קשור למה. העץ הוא התיקיות; הקשתות הן קישורים מפורשים בתוך הרשומות.



אותה קפסולה, שני מבנים: הכלה (ימין) וקישור (שמאל)

למה קבצים ולא מסד-נתונים-גרפי? משום שכל היתרונות של גרף מתקבלים כאן בלי לוותר על מה שקבצים נותנים בחינם: היסטוריה מלאה, השוואת גרסאות, סקירת שינויים, וגיבוי — כל אלה כבר קיימים במערכת ניהול-הגרסאות שהפרויקט ממילא משתמש בה. הגרף הניתן-לשאילתה נבנה מהקבצים בזיכרון בעת הצורך, ואפשר למחוק אותו ולבנות מחדש. הקבצים הם האמת.

מה מתועד בפרויקט

סוגי הרשומות אינם רשימה שרירותית — כל אחד הוא סוג אמת שמתנהג אחרת לאורך זמן. אין ביניהם סוג עבור "פלטפורמה", "שפה" או "רגולציה": מגבלה על הקוד היא **דרישה** שלו, וכשיש לה קוד נפרד היא ממילא **רכיב**. פורמט שמעניק סעיף משלו למימד אחד היה חייב להעניק סעיף לכל מימד.

סוג	מה זה	התכונה שמייחדת אותו
אמנה	החזון, האילוצים, כללי היסוד	נכתב בהתחלה, משתנה לעיתים רחוקות
דרישה	מה המוצר חייב לעשות	חייבת ראיה כדי להיחשב "מתקיימת"
תוכנית	יוזמה שמתפרקת למשימות	נגמרת — בשונה מהפרויקט
החלטה	הכרעה ארכיטקטונית והנימוק לה	לא נמחקת ולא נערכת; מוחלפת בהחלטה חדשה
דיון	שיקולים שנשקלו	שווה שמירה גם בלי הכרעה
תקלה	באג, סיכון, או מטלה	סגירה מחייבת תיקון *ובדיקת-רגרסיה*
תלות	ספריית צד-שלישי + הרישיון שלה	נושאת עובדות שמאפשרות בדיקת-ציות
שחרור	מה יצא, מתי, מאיזה commit	עונה "מה היה ב-2.3" שנים אחרי
תובנה	ידע שנלמד — הנדסי, עיצובי, או צמוד-קוד	לא החלטה ולא משימה; ידע
רכיב	חלק מהמערכת: מודול, תת-מערכת	עונה "מה הארכיטקטורה *עכשיו*"
ממשק	חוזה שאחרים תלויים בו	מחזור-חיים משלו: נוסף · הופחת · הוסר
אבן-דרך	נקודת-זמן שתוכניות מכוונות אליה	תאריך שאפשר לתלות עליו
קו-גרסה	זרם-תחזוקה (x.26 חי, x.25 תיקונים)	מאפשר לתעד backport נכון

שתי דוגמאות ל"עובדה שאינה סקלרית"

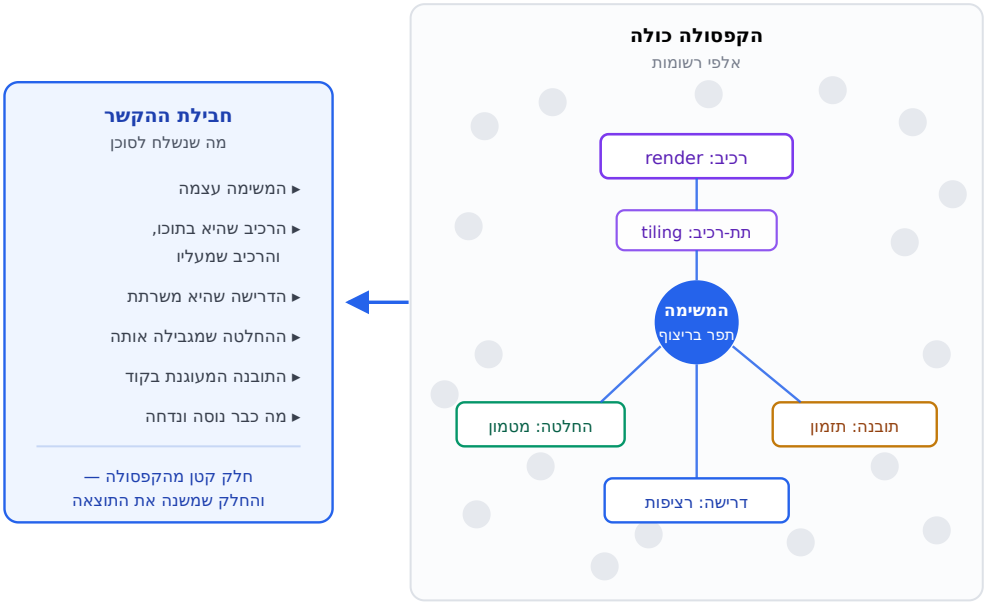
דרישה שמתקיימת חלקית. "מתקיים ב-Windows, לא ב-iPad" אינו ערך אחד. במקום לעגל לתשובה אחת שקרית, הרשומה נושאת תשובה כללית *ורשימת חריגים*, וכל חריג **מצביע על רשומה קיימת** — הדרישה "לרוץ על iPad", או הרכיב שמממש אותה. אין רשימת-מימדים סגורה בפורמט: מוצר משתנה לפי גרסה, לפי תת-מערכת, לפי רגולציה, לפי שפה ולפי נגישות, וכולן כבר רשומות במקום שבו הן חלות.

תיקון שעבר backport. אותו באג תוקן בקו הנוכחי ובקו המתוחזק — בשני commit-ים *שונים*. שדה יחיד יכול לתאר רק אחד מהם, ולכן יש רשימה לפי קו.

> **מה שכל אלה חולקים:** טענה שניתן לבדוק הופכת ל**שדה**, לא למשפט. "אין תקלת חומרה פתוחה", "אין תלות עם רישיון לא-מאושר", "כל דרישת-חובה מאומתת" — כל אלה הופכות לשאלות שמכונה עונה עליהן בשנייה, במקום לסקר ידני לפני כל שחרור.

בניית הקשר לסוכן למשימה אחת

זו השאלה שהפורמט נבנה כדי לענות עליה: סוכן מקבל משימה אחת — למשל "תקן את התפר בריצוף" — ואנחנו רוצים לתת לו את מה שרלוונטי, ורק אותו.



מהזרע החוצה: אבות בעץ + צעדים על קשתות ← חבילת הקשר סגורה

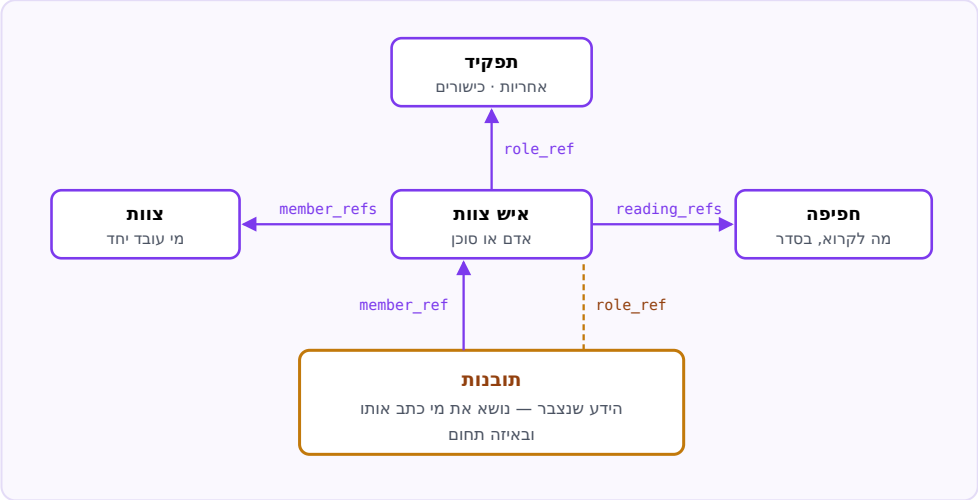
איך זה נקבע — בשלושה כללים

- **האבות בעץ.** המשימה יושבת ברכיב; הרכיב יושב ברכיב-על. אלה ההקשר שבתוכו היא חיה, והם נכנסים תמיד.
- **k צעדים על קשתות, מסוננים לפי סוג.** "מה שמגביל אותי", "מה שאני משרת", "מה שמעוגן בקוד שלי" — נכנסים. "מי עוד תלוי בי" יכול להישאר בחוץ אם אינו רלוונטי למשימה.
- **תקציב.** אם החבילה גדולה מדי, חותכים לפי מרחק — לא באקראי.

> **למה זה חייב להיות מבני ולא "חיפוש חכם":** חיפוש-דמיון סמנטי מדרג לפי דמיון ניסוח. החלטה שמגבילה את המשימה עשויה להיות מנוסחת בלי אף מילה משותפת איתה — והיא בדיוק מה שאסור לפספס. לכן **המבנה בוחר** מה נכנס (שלמות מובטחת), ודמיון סמנטי לכל היותר **מדרג** בתוך מה שכבר נבחר. חבילה שהשמיטה בשקט החלטה מגבילה שגויה באופן שציון-דמיון לא יכול לזהות.

הצוות והחברה: אנשים, תפקידים ותובנות

לחברה יש אמת משלה, שאינה שייכת לאף פרויקט: מי עובד כאן, מה כל תפקיד אחראי לו, ומה הארגון למד. זהו **פורמט הארגון** — אותו דקדוק בדיוק, סוגי רשומות אחרים, ובית נפרד.



פורמט הארגון — התובנה מחוברת גם למי שכתב אותה וגם לתחום שלה

התכונה החשובה: הידע שורד את התחלופה

כשאיש צוות עוזב, הרשומה שלו לא נמחקת — היא עוברת למצב "עזב". **התובנות שהוא כתב נשארות**, וממשיכות להיות מחוברות לתחום שלהן. זה ההבדל בין ארגון שבו הידע יושב בראש של אנשים לבין ארגון שבו הוא נכס.

ומכאן נובעת החפיפה: רשומת-חפיפה היא רשימת קריאה מסודרת שנבנית מהידע שכבר נאסף. מי שנכנס לתפקיד מקבל את מה שקודמו למד, לא דף ריק. זה נכון באותה מידה לאדם חדש ולסוכן שמתחיל משמרת — שניהם "אנשי צוות" בפורמט, ובמכוון.

שתי רמות של ידע

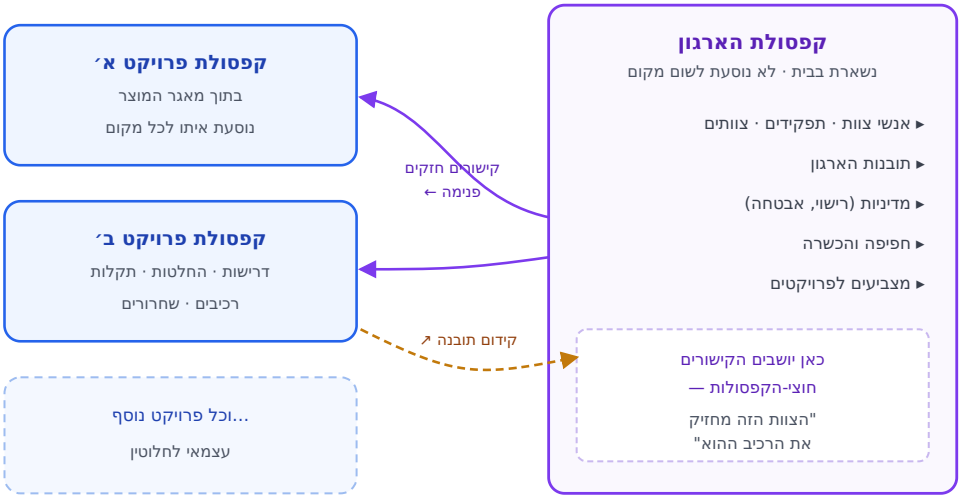
תובנה של תפקיד. "כשבודקים ביצועים של רינדור, המדד שמטעה הוא ממוצע ולא אחוזון 95." שייכת למקצוע — ורלוונטית לכל מי שימלא את התפקיד, בכל פרויקט.

תובנה של החברה. "החלטות רישוי נסגרות מול היועץ המשפטי לפני ולא אחרי הבחירה בספרייה." חוצה תפקידים ופרויקטים — ולכן היא של הארגון כולו.

> **הכלל שמונע כפילות:** לכל עובדה יש **בית אחד**. תובנה לא מועתקת בין רשומות — היא נכתבת פעם אחת, ומי שצריך אותה *מצביע* עליה. שתי גרסאות של אותה עובדה הן שתי גרסאות שיסתרו זו את זו ביום שאחת מהן תתעדכן.

היחס בין תיעוד החברה לתיעוד הפרויקטים

שתי קפסולות, שני בתים — ובכוונה. ההפרדה אינה סדר לשם סדר; היא נובעת משלושה גבולות אמיתיים.



הפרויקט נוסע; הארגון נשאר. הקישורים החזקים מצביעים פנימה.

שלושת הגבולות

- **גבול ניידות.** קפסולת-פרויקט נוסעת עם המאגר — clone, fork, מסירה ללקוח או לקבלן. אם ידע החברה יושב בתוכה, הוא נוסע איתה.
- **גבול מחזור-חיים.** פרויקט מגיע לארכיון; החברה לא. ידע ארגוני חייב לשרוד את ארכוב הפרויקט שבו נלמד.
- **גבול סמכות.** מדיניות ורוסטר אינם אמורים להיות ניתנים לעריכה על-ידי מי שעובד על פרויקט מסוים.

ואיך בכל זאת מצליבים ביניהם

כתובת חוצת-קפסולות. קישור פנימי **חייב** להצביע על רשומה קיימת; קישור חיצוני **רשאי** להישאר בלתי-פתיר — וזו בדיוק ערובת הניידות: קפסולת-פרויקט נשארת תקפה וקריאה גם כשאף קפסולה אחרת אינה מחוברת. מכאן נובע שהקישורים החזקים יושבים בצד שנשאר בבית.

> **ומה עם תובנה שנולדה בפרויקט והתבררה כארגונית?** היא **עוברת** לקפסולת הארגון, ובמקומה נשארת "מצבה" — רשומה קצרה שאומרת לאן היא הלכה. לא העתקה: בית אחד לכל עובדה, וההיסטוריה לא נקטעת.

מה נבדק אוטומטית — ומה קפסה איננה

לפורמט נלווה בודק קריאה-בלבד. הוא לא מתקן ולא כותב — הוא אומר מה לא תקין, בשפה שגם תוכנה יכולה לפעול לפיה.

נבדק	מה זה מונע בפועל
דרישה "מתקיימת" ללא ראיה	סטטוס שמישהו עדכן בתקווה
באג סגור ללא תיקון ובדיקת-רגרסיה	באג שיחזור ואיש לא ידע שכבר היה
קישור פנימי שמצביע לשומקום	גרף שנראה שלם ואינו — הסכנה האמיתית
חריג שמצביע על רשומה שאינה קיימת	שגיאת כתיב שבשקט לא אומרת כלום
קישור שחוזר על מה שהנתיב כבר אומר	קשת מיותרת שצריך לתחזק לנצח
ממשק "הוסר" בלי תאריך הסרה	צרכן שמגלה את ההסרה בדרך הקשה
תלות עם רישיון אסור ללא החלטה מאשרת	חשיפה משפטית שאיש לא הכריע עליה

כל ממצא נושא **קוד יציב** ו**דרגת חומרה**. הקוד הוא מה שתוכנה פועלת לפיו; הניסוח לאדם עשוי להשתנות. אזהרה אינה פוסלת קפסולה — היא מציינת דבר שכדאי, ולא דבר שחייבים.

מה קפסה איננה — במכוון

- **לא כלי ניהול משימות.** אין "מי עובד עכשיו", אין שעות, אין לוח פעיל. זה מצב-ריצה, והוא נכון לרגע.
- **לא מדדים.** כל מה שנגזר מהרשומות — ספירות, לוחות-זמנים, דוחות — מחושב בעת הצורך ולא נשמר.
- **לא קוד.** הקפסולה מתארת את הפרויקט; המאגר סביבה מחזיק את המוצר.
- **לא מתקנת את עצמה.** הבודק קורא בלבד. תיקון הוא פעולת תחזוקה — ושייך לכלי שמפעילים, לא לפורמט.

> **בשורה אחת:** קפסה היא הזיכרון הדורבני של הפרויקט ושל הארגון, שמור כקבצים קריאים שאיש אינו תלוי בכלי כדי לקרוא — ומקושר מספיק כדי שאפשר יהיה לשלוף ממנו בדיוק את מה שרלוונטי למשימה אחת, במקום להטביע את מי שעובד עליה בכל מה שאי-פעם נכתב.